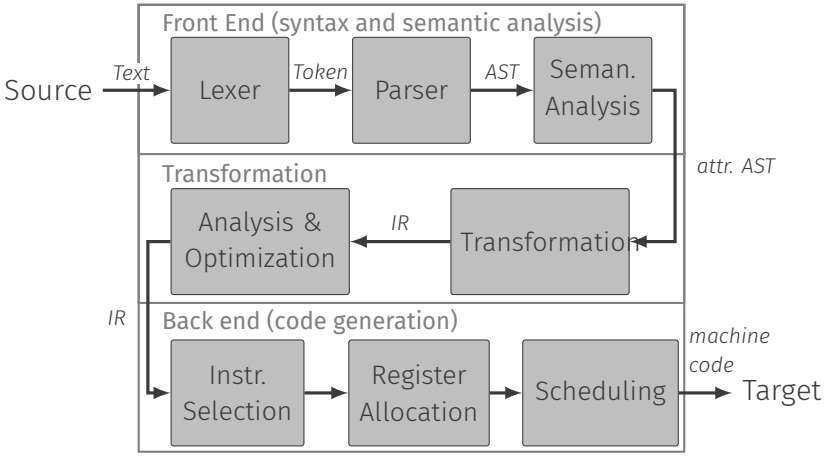
GEORG-AUGUST-UNIVERSITÄT
GÖTTINGEN IN PUBLICA COMMODA SEIT 1737

# Semantic Analysis

Roland Leißa (based on slides by S. Hack)

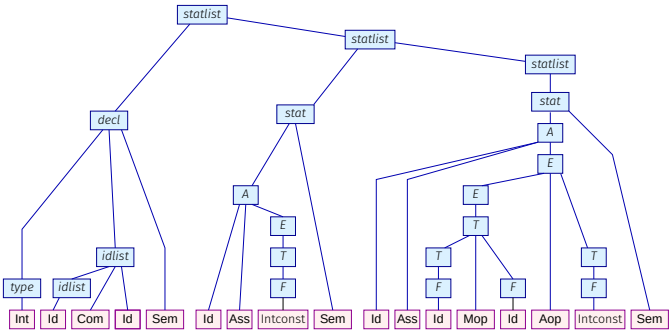
University of Göttingen

# Overview


Front End (syntax and semantic analysis)
Source
Text
Lexer
Token
Parser
AST
Seman. Analysis
attr. AST
Transformation
Analysis & Optimization
IR
Transformation
IR
Back end (code generation)
Instr. Selection
Register Allocation
Scheduling
machine code
Target

# Abstract Syntax Trees (AST)

- Delete Terminals that were only there to make the grammar unambiguous
- Remove Nonterminals in chain productions from parse tree (below) (typical if precedence and associativity were expressed in grammar)




```
int x, y;
x = 42;
y = x * x + 23;
```

# Semantic Analysis

Many programming languages have a static semantics.
(Interpretation of the syntax independent of the inputs.)

Purpose:

- Refute at compile-time programs that potentially get stuck
- Infer information about program helpful for compiler

Program analyses with respect to the static semantics are often called semantic analyses. Commonly, we have two of them:

- Name Analysis
- Type Checking

# Do we need semantic analysis?

Consider the following Python program:

```
def f(x):
    if x == 0:
        return x
    else:
        return y
```

What happens on `f(0)` and `f(1)`?

# Static vs Dynamic Semantics

```
class Foo {
    void method() {
        if (false) bar(23);
    }

    void bar(int x, int y) { /*...*/ }
}
```

# Do we need semantic analysis?

Python does not have a static semantics, hence it has to “rule out” many invalid programs at runtime:

- When an identifier is declared, enter it into a definition table
- When it is used, look up, if there is a value for that identifier
- Incurs significant runtime overhead
- Allows more flexible (really really really?) code: i.e. can add fields to objects at runtime

Similar things happen because of “duck typing”:

- Resolve operator overloading at runtime: `a + b`
  Do we add two ints? Floats? Append to a string? Or is a an object of a class that redefines +?
- Similarly for methods: `a.b(c)`
  Does the class of `a` really provide a method `b` or a superclass?

# Limits of static semantics

- C leaves semantics of many programs (under certain inputs) undefined. Result: runtime errors, e.g. division by zero, dereference of non-allocated memory
- Static semantics cannot capture them: in general undecidable
- Make language more restrictive to avoid some of them. E.g. Java:
  - Cannot take address of a field or array cell
  - Garbage collection policy avoids dangling pointers
- “Totalize” behavior, e.g.
  - Throw exception on `null` dereference
  - Array out-of-bounds check
  - Incurs runtime overhead!

# Name Analysis

- Every entity (variable, function, data type, etc.) has a defining occurrence (definition) that gives it a name and describes its properties with respect to static semantics (e.g. type)
- Every defining occurrence has a scope in which it is valid
- Scope is the static pendant to lifetime of referred object (at runtime)
- An applied occurrence refers to a definition

Purpose of name analysis:

- Ensure that every applied occurrence refers to a unique defining occurrence
- Relate applied occurrence to defining occurrence in AST

Issues: Validity, Visibility, Qualification, Namespaces, Overloading

# Validity

The scope (of validity) of a definition of an identifier *x* is the part of the program (nodes in the AST) in which an applied occurrence of *x* refers to it.

Inside a scope, each identifier can only defined once.

## Example

```
{
    int y;
    // ...
    int x = 1;
    // ...
    y = x + 1;
}
```

The definition of x is valid inside the entire block. But only visible after its definition.

# Visibility

Inside its scope, a definition of an identifier *x* can be invisible (hidden, overwritten). Then, it is still forbidden to redefine *x* in the same scope (but in a nested one!) but not possible to refer to that definition:

```
for (int i = 0; i < n; i++)
    for (int i = 0; i < m; i++)
        A[i] = ...; // app occ of i refers
                    // to innermost definition
```

Possible in C. Java forbids overwriting definitions of local variables but allows

```
class X {
    int x;
    void foo(int x) {
        x = 1;
    }
}
```

# Qualification

Some entities are composed of other entities (e.g. modules, classes, structs). Their definition opens a scope in which their constituents are defined:

```
struct vec3_t {
    float x, y, z;
};
```

The scope of `x`, `y`, and `z` is limited by `{ }`.

Use qualification to refer to constituents:

```
vec3_t v;
// ...
v.x = 1.0f;
```

In the static semantics, the `.` makes the identifier on the right-hand side refer to its definition inside the scope of the entity described by the left-hand side.

# Multiple Name Spaces

Many programming languages allow multiple name spaces. More than one definition of the same identifier can be valid and visible at the same program point. Must be clear from the context which entity it is referred to.

## Example (C)

labels, structs/unions, and functions/variables are in disjoint name spaces.

```c
struct x {
    int x;
};

int foo(int x) {
    struct x y; // x refers to struct
    y.x = x;    // refers to parameter
    goto x;     // refers to label
x:
    return y.x; // refers to parameter
}
```

# AST Design - Bases

```
class ASTNode {
public:
    ASTNode(Loc loc)
        : loc_(loc)
    {}
    virtual ~ASTNode() {}

    Loc loc() const { return loc_; }
    void dump() const { stream(std::cout) << std::endl; }
    virtual std::ostream& stream(std::ostream& o) const = 0;

private:
    Loc loc_;
};

class Stmt : public ASTNode {
public:
    Stmt(Loc loc)
        : ASTNode(loc)
    {}

    virtual void check(Sema&) = 0;
};
```

```
class Expr : public ASTNode {
public:
    Expr(Loc loc)
        : ASTNode(loc)
    {}

    const Type* type() const { return type_; }
    virtual const Type* check(Sema&) = 0;

protected:
    const Type* type_ = nullptr;
};
```

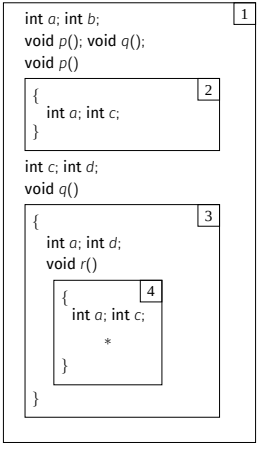
# AST Design - Stmt/Expr

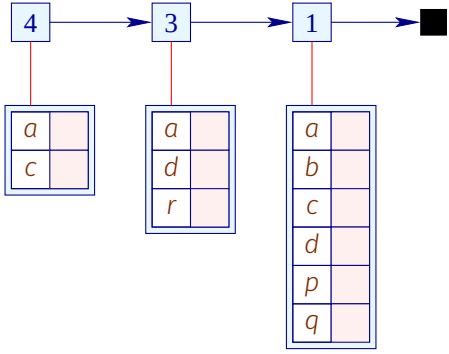
```
class IfStmt : public Stmt {
public:
    IfStmt(Loc loc, Ptr<Expr>&& cond,
           Ptr<Stmt>&& cons, Ptr<Stmt>&& alt)
        : Stmt(loc)
        , cond_(std::move(cond))
        , cons_(std::move(cons))
        , alt_(std::move(alt))
    {}

    const Expr* cond() const { return cond_.get(); }
    const Stmt* cons() const { return cons_.get(); }
    const Stmt* alt() const { return alt_.get(); }

    void check(Sema&) override;
    //...

private:
    Ptr<Expr> cond_;
    Ptr<Stmt> cons_;
    Ptr<Stmt> alt_;
};
```

```
class BinExpr : public Expr {
public:
    BinExpr(Loc loc, Ptr<Expr>&& lhs, Tok::Tag tag,
                     Ptr<Expr>&& rhs)
        : Expr(loc)
        , tag_(tag)
        , lhs_(std::move(lhs))
        , rhs_(std::move(rhs))

    Tok::Tag tag() const { return tag_; }
    const Expr* lhs() const { return lhs_.get(); }
    const Expr* rhs() const { return rhs_.get(); }

    const Type* check(Sema&) override;
    //...

private:
    Tok::Tag tag_;
    Ptr<Expr> lhs_;
    Ptr<Expr> rhs_;
};
```

# Symbol (Definition) Tables

To detect the valid defining occurrence for an applied occurence and to handle visibility one typically uses a stack of environments (Env = Id → Type).




Pink boxes are pointers to the AST node of the identifier's declaration.

# Scoping

```
void Decl::check(Sema& sema) {
    // ...
    sema.add(this);
    // ...
}

void CompoundStmt::check(Sema& sema) {
    sema.push();
    for (auto&& stmt : stmts)
        stmt->check(sema);
    sema.pop();
}

const Type* IdExpr::check(Sema& sema) {
    if (auto decl = sema.lookup(sym())) {
        decl_ = decl;
        return type_ = decl->type();
    }

    loc().err() << "identifier not declared" << std::endl;
    return type_ = sema.error_type();
}
```

```
class IdExpr : public Expr {
public:
    IdExpr(Loc loc, Sym sym)
        : Expr(loc)
        , sym_(sym)
    {}

    Sym sym() const { return sym_; }
    const Decl* decl() const { return decl_; }
    const Type* check(Sema&) override;

private:
    Sym sym_;
    Decl* decl_ = nullptr;
};
```

# Type Checking C

```
const Type* BinExpr::check(Sema& sema) {
    auto ltype = lhs()->check(sema);
    auto rtype = rhs()->check(sema);

    switch (tag()) {
        case Tok::Tag::O_Add: {
            if (dynamic_cast<const ArithType*>(ltype) && dynamic_cast<const ArithType*>(rtype)) {
                return type_ = sema.int_type();
            } else if (auto ptr = dynamic_cast<const PtrType*>(ltype)) {
                if (dynamic_cast<const IntType*>(rtype) && ptr->pointee()->isCompleteObjType())
                    return type_ = ptr;
            } else if (auto ptr = dynamic_cast<const PtrType*>(rtype)) {
                if (dynamic_cast<const IntType*>(ltype) && ptr->pointee()->isCompleteObjType())
                    return type_ = ptr;
            }

            loc().err() << "invalid operand types for addition" << std::endl;
            return type_ = sema.error_type();
        }
        // ...
    }
}
```

# The Simply Typed $\lambda$-Calculus

$$
\begin{aligned}
V &\to \lambda x : T.E \\
&\mid \texttt{tt} \\
&\mid \texttt{ff}
\end{aligned}
$$

$$
\begin{aligned}
E &\to V \\
&\mid x \\
&\mid EE \\
&\mid \texttt{if } E \texttt{ then } E \texttt{ else } E
\end{aligned}
$$

$$
\begin{aligned}
T &\to \texttt{bool} \\
&\mid T \to T
\end{aligned}
$$

$$
\begin{aligned}
\Gamma &\to \varepsilon \\
&\mid \Gamma, x : T
\end{aligned}
$$

# Static Semantics

$\boxed{\Gamma \vdash E : T}$

$$\textsc{Var}\ \frac{x : T \in \Gamma}{\Gamma \vdash x : T} \qquad \textsc{Abs}\ \frac{\Gamma, x : T \vdash E : T'}{\Gamma \vdash \lambda x : T.E : T \to T'} \qquad \textsc{App}\ \frac{\Gamma \vdash E_1 : T_1 \to T_2 \qquad \Gamma \vdash E_2 : T_1}{\Gamma \vdash E_1\ E_2 : T_2}$$

$$\textsc{TT}\ \frac{}{\Gamma \vdash \texttt{tt} : \texttt{bool}} \qquad \textsc{FF}\ \frac{}{\Gamma \vdash \texttt{ff} : \texttt{bool}}$$

$$\textsc{If}\ \frac{\Gamma \vdash E_c : \texttt{bool} \qquad \Gamma \vdash E_t : T \qquad \Gamma \vdash E_f : T}{\Gamma \vdash \texttt{if}\ E_c\ \texttt{then}\ E_t\ \texttt{else}\ E_f : T}$$

# Dynamic Semantics

$\boxed{E \to E}$

$$\textsc{App}_1 \frac{E_1 \to E_1'}{E_1\ E_2 \to E_1'\ E_2} \qquad \textsc{App}_2 \frac{E_2 \to E_2'}{V_1\ E_2 \to V_1 E_2'} \qquad \beta \frac{}{(\lambda x : T.E)\ V \to E[V/x]}$$

$$\textsc{If} \frac{E_c \to E_c'}{\texttt{if}\ E_c\ \texttt{then}\ E_t\ \texttt{else}\ E_f \to \texttt{if}\ E_c'\ \texttt{then}\ E_t\ \texttt{else}\ E_f}$$

$$\textsc{If}_t \frac{}{\texttt{if tt then}\ E_t\ \texttt{else}\ E_f \to E_t} \qquad \textsc{If}_f \frac{}{\texttt{if ff then}\ E_t\ \texttt{else}\ E_f \to E_f}$$

# Type Safety

*Well typed programs cannot go wrong.* – *Robin Milner*

**Progress**

A well-typed expression is not stuck
(either it is a value or it can take a step according to the evaluation rules).

$$\text{If } \Gamma \vdash E : T, \text{ then } E \in V \text{ or } \exists E' : E \to E'.$$

**Preservation**

If a well-typed expression takes a step of evaluation, then the resulting expression is also well typed.

$$\text{If } \Gamma \vdash E : T \text{ and } E \to E', \text{ then } \Gamma \vdash E' : T.$$

Type Safety = Progress + Preservation aka Soundness

# Type Safety in Programming Languages

## Type Safe

- Simply-Typed $\lambda$-Calculus
- Standard ML (some extensions provide unsafe features though)

## Not Type Safe

- C/C++ (undefined behavior)
- Most languages have at least some unsafe features.

## Java

*Java is not type-safe, though it was intended to be.*

`https://www.cis.upenn.edu/~bcpierce/courses/629/papers/Saraswat-javabug.html`

# Type Inference

Often types of variables are uniquely identified by the way the variables are used. Use this information to avoid declarations and infer types automatically.

This is common in functional language and modern functional/imperative languages such as Scala, Rust, OCaml, ...

Consider following example:

```
let rec fac = fun -> if x <= 0 then 1
                     else x * fac (x - 1)
```

Because one branch uses the int constant 1, the term `x * fac (x - 1)` must be an int as well. Hence `x` and `fac (x - 1)` must be ints as well. And finally `fac` is a function from `int -> int`.